

# **EXPERIMENT-16**

1. Algorithm:

1. Start.

2. Declare an integer variable choice.

3. Display the following menu:

1 — Check digits in increasing order.

2 — Check digits in decreasing order.

3 — Find largest and smallest digit.

4 — Find second largest and second smallest digit.

4. Read the user's choice.

5. Use switch(choice).

6. If choice is 1, call increasingOrder().

7. If choice is 2, call decreasingOrder().

8. If choice is 3, call largestSmallest().

9. If choice is 4, call secondLargestSmallest().

10. Otherwise, display "Invalid choice".

11. Stop.

### Function 1 — Increasing Order:

1. Read a positive integer.
2. Find the divisor corresponding to its leftmost digit.
3. Store the first digit as previous.
4. Extract the remaining digits one by one from left to right.
5. Compare previous with current.
6. If  $\text{previous} \geq \text{current}$ , the digits are not in strictly increasing order.
7. Otherwise, continue checking.
8. Display the result.

Example: 12357 → Increasing

Example: 12435 → Not increasing

### Function 2 — Decreasing Order:

1. Read a positive integer.
2. Find the divisor corresponding to its leftmost digit.
3. Store the first digit as previous.
4. Extract the remaining digits from left to right.
5. Compare previous with current.
6. If  $\text{previous} \leq \text{current}$ , the digits are not in strictly decreasing order.

7. Otherwise, continue checking.

8. Display the result.

Example: 97531 → Decreasing

Example: 97481 → Not decreasing

Function 3 — Largest and Smallest Digit:

1. Read a positive integer.
2. Initialize largest = 0 and smallest = 9.
3. Extract the last digit using  $n \% 10$ .
4. If the digit is greater than largest, update largest.
5. If the digit is smaller than smallest, update smallest.
6. Remove the last digit using  $n = n / 10$ .
7. Repeat until all digits are processed.
8. Display the largest and smallest digits.

Function 4 — Second Largest and Second Smallest Digit:

Here, second largest/smallest means the second distinct value. For example, in 99821, the second largest is 8, not another 9.

1. Read a positive integer.
2. Initialize:  
largest = -1  
secondLargest = -1

smallest = 10  
secondSmallest = 10

3. Extract each digit.
4. Compare it with largest and secondLargest and update them when necessary.
5. Compare it with smallest and secondSmallest and update them when necessary.
6. Ignore duplicate values while determining the second largest/smallest.
7. Repeat for all digits.
8. Display the second largest and second smallest digits.
9. If they do not exist, display an appropriate message

## 2. Test Case Table:

| Case | Choice | Number | Expected Output                         | Actual Output                           | Result |
|------|--------|--------|-----------------------------------------|-----------------------------------------|--------|
| 1    | 1      | 12357  | in increasing order                     | in increasing order                     | Pass   |
| 2    | 1      | 12435  | not in increasing order                 | not in increasing order                 | Pass   |
| 3    | 2      | 97531  | in decreasing order                     | in decreasing order                     | Pass   |
| 4    | 2      | 97481  | not in decreasing order                 | not in decreasing order                 | Pass   |
| 5    | 3      | 50291  | Largest = 9, Smallest = 0               | Largest = 9, Smallest = 0               | Pass   |
| 6    | 3      | 73542  | Largest = 7, Smallest = 2               | Largest = 7, Smallest = 2               | Pass   |
| 7    | 4      | 583172 | Second largest = 7, Second smallest = 2 | Second largest = 7, Second smallest = 2 | Pass   |
| 8    | 4      | 96241  | Second largest = 6, Second smallest = 2 | Second largest = 6, Second smallest = 2 | Pass   |

### 3. Program, Compilation and Execution:



```
GNU nano 9.0
#include <stdio.h>

/* Function declarations */
void increasingOrder(void);
void decreasingOrder(void);
void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;

    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            increasingOrder();
            break;

        case 2:
            decreasingOrder();
            break;

        case 3:
            largestSmallest();
            break;

        case 4:
            secondLargestSmallest();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```



The image shows a terminal window with the GNU nano 9.0 text editor. The editor's interface includes a top bar with icons for file operations and a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main area displays a C program that checks if the digits of a positive integer are in increasing order. The code uses a while loop to extract digits from the number and compares them to determine if they are in increasing order. The program prompts the user to enter a positive integer and prints the result.

```
/* Function to check increasing order */
void increasingOrder(void)
{
    long long n, temp, divisor = 1;
    int previous, current;
    int increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    temp = n;

    /* Find divisor for leftmost digit */
    while (temp ≥ 10)
    {
        divisor = divisor * 10;
        temp = temp / 10;
    }

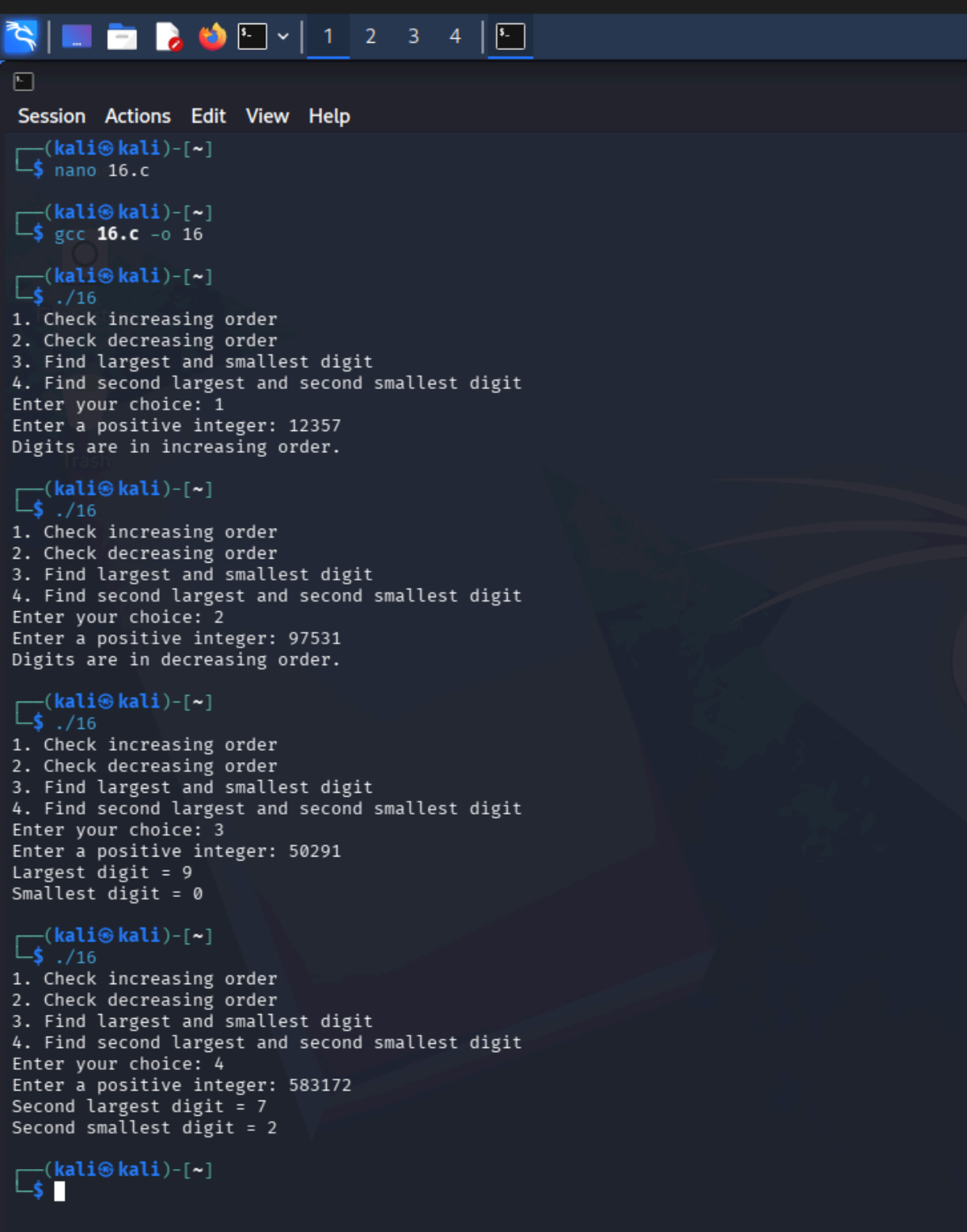
    previous = n / divisor;
    n = n % divisor;
    divisor = divisor / 10;

    while (divisor > 0)
    {
        current = n / divisor;

        if (previous ≥ current)
        {
            increasing = 0;
            break;
        }

        previous = current;
        n = n % divisor;
        divisor = divisor / 10;
    }

    if (increasing == 1)
        printf("Digits are in increasing order.\n");
    else
        printf("Digits are not in increasing order.\n");
}
```



```
(kali㉿kali)-[~]
$ nano 16.c

(kali㉿kali)-[~]
$ gcc 16.c -o 16

(kali㉿kali)-[~]
$ ./16
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12357
Digits are in increasing order.

(kali㉿kali)-[~]
$ ./16
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97531
Digits are in decreasing order.

(kali㉿kali)-[~]
$ ./16
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
Enter a positive integer: 50291
Largest digit = 9
Smallest digit = 0

(kali㉿kali)-[~]
$ ./16
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 583172
Second largest digit = 7
Second smallest digit = 2

(kali㉿kali)-[~]
$
```

#### 4. Conclusion:

This follows the requirement of four separate void functions with no parameters and simple function calls from the switch-case.

**NAME: TAANSIH AGARWAL**

**BATCH: B-15**

**SAP ID: 590034662**